EE 446 TinyML — Model Pruning with Quantization

Student TODO Version: Pruning and Quantization of a DNN Using the UCI Human Activity Recognition Dataset

Overview

In this notebook, you will:

- train a baseline DNN on the **UCI HAR** dataset,
- apply **magnitude-based pruning**,
- compare the pruned model before and after `strip_pruning(...)`, and
- combine pruning with **float16 quantization**.

Use the **`Python (tinyml-arduino)`** Jupyter kernel for this notebook.

1. Environment Setup

This notebook is designed to run with the **`Python (tinyml-arduino)`** Jupyter kernel that you already created.

This notebook assumes the environment already contains:

- `tensorflow==2.14.1`
- `tensorflow-model-optimization==0.8.0`
- `numpy`, `pandas`, `matplotlib`, and `scikit-learn`

Do **not** reinstall TensorFlow packages inside the notebook if you are already using the working TinyML environment.

In [1]:
```
import os
import math
import zipfile
import random
import urllib.request
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
import tensorflow_model_optimization as tfmot

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
from tensorflow import keras
from tensorflow.keras import layers

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("Python executable:", os.sys.executable)
print("TensorFlow version:", tf.__version__)
print("TF-MOT version:", tfmot.__version__)
```

```
2026-04-23 21:56:56.798291: I tensorflow/core/util/port.cc:111] oneDNN custom operat
ions are on. You may see slightly different numerical results due to floating-point
round-off errors from different computation orders. To turn them off, set the enviro
nment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-04-23 21:56:56.801632: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find
cuda drivers on your machine, GPU will not be used.
2026-04-23 21:56:56.831559: E tensorflow/compiler/xla/stream_executor/cuda/cuda_dnn.
cc:9342] Unable to register cuDNN factory: Attempting to register factory for plugin
cuDNN when one has already been registered
2026-04-23 21:56:56.831598: E tensorflow/compiler/xla/stream_executor/cuda/cuda_fft.
cc:609] Unable to register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
2026-04-23 21:56:56.831615: E tensorflow/compiler/xla/stream_executor/cuda/cuda_bla
s.cc:1518] Unable to register cuBLAS factory: Attempting to register factory for plu
gin cuBLAS when one has already been registered
2026-04-23 21:56:56.839151: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find
cuda drivers on your machine, GPU will not be used.
2026-04-23 21:56:56.841111: I tensorflow/core/platform/cpu_feature_guard.cc:182] Thi
s TensorFlow binary is optimized to use available CPU instructions in performance-cr
itical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operati
ons, rebuild TensorFlow with the appropriate compiler flags.
2026-04-23 21:56:57.586729: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38]
TF-TRT Warning: Could not find TensorRT
```

```
Python executable: /home/redey/ai/projects/tinyml-arduino/bin/python
TensorFlow version: 2.14.1
TF-MOT version: 0.8.0
```

2. Download and Extract the UCI HAR Dataset

The original dataset contains:

- **561 numerical features** extracted from smartphone sensor signals,
- **6 activity classes**, and
- predefined **training** and **test** splits.

The code below downloads and extracts the dataset if it is not already present in the working directory.

In [2]:
```
dataset_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00240/UCI%
zip_path = "uci_har_dataset.zip"
```

```
extract_dir = "."

if not os.path.exists("UCI HAR Dataset"):
    urllib.request.urlretrieve(dataset_url, zip_path)
    with zipfile.ZipFile(zip_path, "r") as zf:
        zf.extractall(extract_dir)
    print("Dataset downloaded and extracted.")
else:
    print("Dataset directory already exists.")
```

```
Dataset directory already exists.
```

3. Load the Data

In [3]:

```
def load_har_data(root_dir="UCI HAR Dataset"):
    # TODO:
    # 1. Load X_train from train/X_train.txt as float32
    # 2. Load y_train from train/y_train.txt as int32
    # 3. Load X_test from test/X_test.txt as float32
    # 4. Load y_test from test/y_test.txt as int32
    # 5. Convert labels to zero-based class indices by subtracting 1

    # <-- Enter your code here <--#
    # Load feature matrices as float32.
    X_train = np.loadtxt(os.path.join(root_dir, "train", "X_train.txt"), dtype=np.f
    X_test = np.loadtxt(os.path.join(root_dir, "test", "X_test.txt"), dtype=np.floa

    # Load labels as int32 and convert from 1-based labels to 0-based class indices
    y_train = np.loadtxt(os.path.join(root_dir, "train", "y_train.txt"), dtype=np.i
    y_test = np.loadtxt(os.path.join(root_dir, "test", "y_test.txt"), dtype=np.int3

    return X_train, y_train, X_test, y_test

X_train, y_train, X_test, y_test = load_har_data()

class_names = [
    "WALKING",
    "WALKING_UPSTAIRS",
    "WALKING_DOWNSTAIRS",
    "SITTING",
    "STANDING",
    "LAYING"
]

# TODO:
# Define num_features and num_classes.
# Print the train/test shapes and the number of features/classes.

# <-- Enter your code here <--#
num_features = X_train.shape[1]
num_classes = len(class_names)

print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
```

```
print("y_test shape:", y_test.shape)
print("Number of features:", num_features)
print("Number of classes:", num_classes)
```

```
X_train shape: (7352, 561)
y_train shape: (7352,)
X_test shape: (2947, 561)
y_test shape: (2947,)
Number of features: 561
Number of classes: 6
```

4. Quick Inspection

In [4]:

```
# TODO:
# Create a small summary table showing:
# - class index,
# - class name, and
# - number of training samples in each class.

# <-- Enter your code here <--#
class_counts = np.bincount(y_train, minlength=num_classes)

class_summary = pd.DataFrame({
    "Class Index": np.arange(num_classes),
    "Class Name": class_names,
    "Training Samples": class_counts
})

class_summary
```

Out[4]:

	Class Index	Class Name	Training Samples
0	0	WALKING	1226
1	1	WALKING_UPSTAIRS	1073
2	2	WALKING_DOWNSTAIRS	986
3	3	SITTING	1286
4	4	STANDING	1374
5	5	LAYING	1407

5. Train a Baseline DNN

We will use a compact dense neural network that is appropriate for a numerical-feature TinyML workflow.

Architecture

- Input: 561 features

- Dense(256, ReLU)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(6, Softmax)

In [5]:
```
def build_baseline_model(input_dim, num_classes):
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(256, activation="relu"),
        layers.Dense(128, activation="relu"),
        layers.Dense(64, activation="relu"),
        layers.Dense(num_classes, activation="softmax")
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

    return model

baseline_model = build_baseline_model(num_features, num_classes)
baseline_model.summary()
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense (Dense)               (None, 256)               143872

 dense_1 (Dense)             (None, 128)               32896

 dense_2 (Dense)             (None, 64)                8256

 dense_3 (Dense)             (None, 6)                 390

=================================================================
Total params: 185414 (724.27 KB)
Trainable params: 185414 (724.27 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

Train the Baseline Model

In [6]:
```
callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        patience=5,
        restore_best_weights=True
    )
]

# TODO:
```

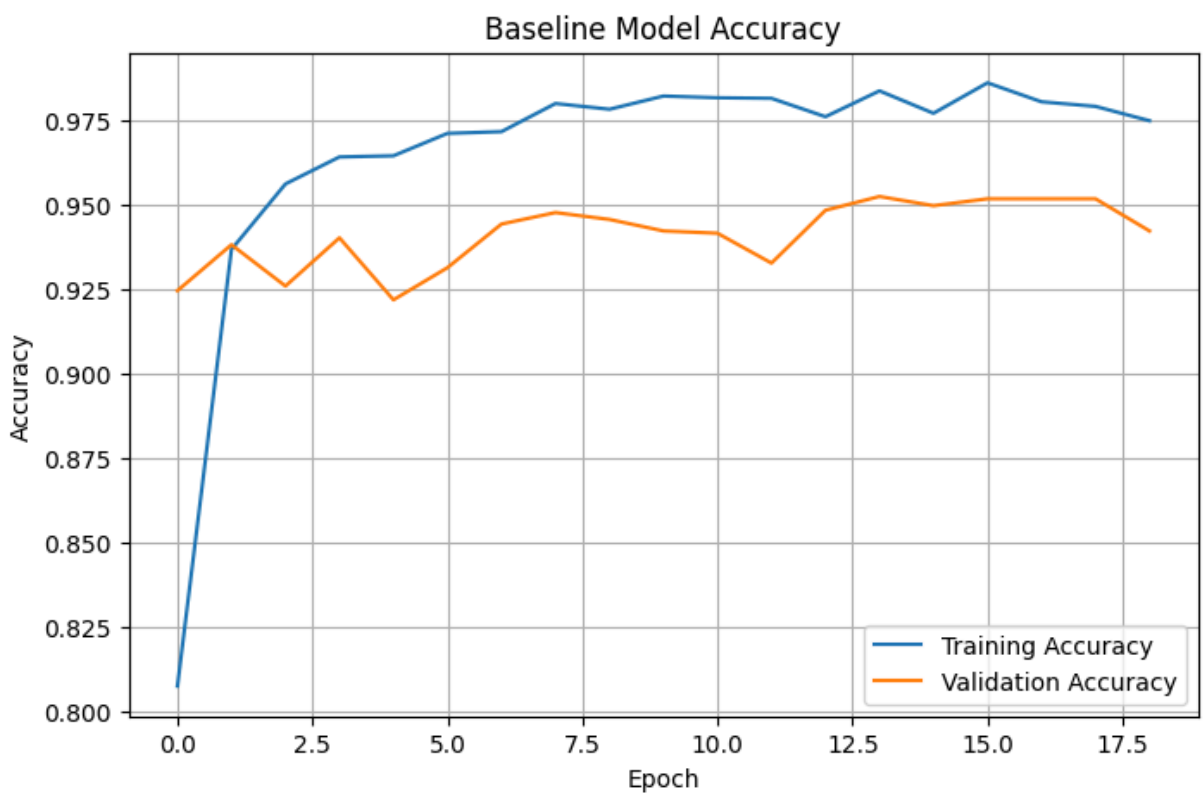
```
# Train the baseline model using:
# - validation_split=0.2
# - epochs=40
# - batch_size=64
# - callbacks=callbacks

# <-- Enter your code here <--#
history = baseline_model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=40,
    batch_size=64,
    callbacks=callbacks,
    verbose=1
)
```

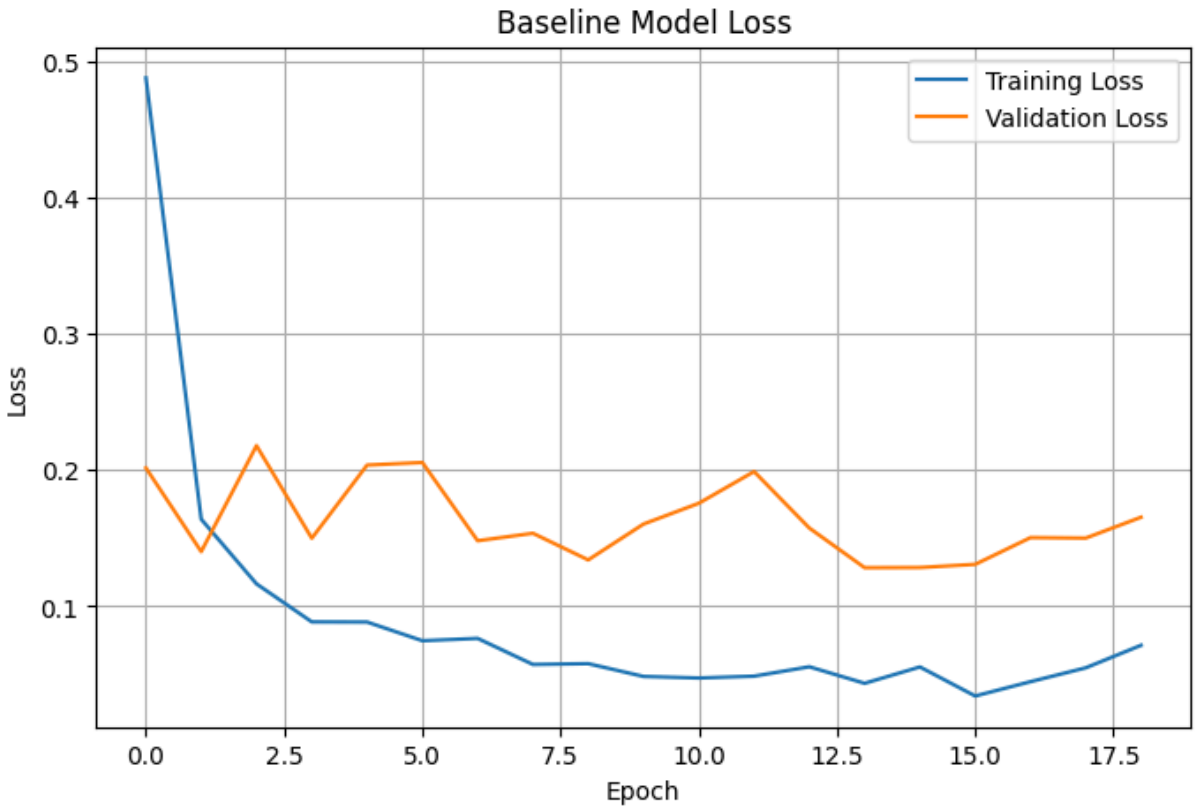
```
Epoch 1/40
92/92 [==============================] - 1s 6ms/step - loss: 0.4880 - accuracy: 0.80
75 - val_loss: 0.2016 - val_accuracy: 0.9245
Epoch 2/40
92/92 [==============================] - 0s 4ms/step - loss: 0.1640 - accuracy: 0.93
67 - val_loss: 0.1402 - val_accuracy: 0.9381
Epoch 3/40
92/92 [==============================] - 0s 4ms/step - loss: 0.1165 - accuracy: 0.95
61 - val_loss: 0.2180 - val_accuracy: 0.9259
Epoch 4/40
92/92 [==============================] - 0s 4ms/step - loss: 0.0886 - accuracy: 0.96
41 - val_loss: 0.1500 - val_accuracy: 0.9402
Epoch 5/40
92/92 [==============================] - 0s 4ms/step - loss: 0.0885 - accuracy: 0.96
45 - val_loss: 0.2037 - val_accuracy: 0.9218
Epoch 6/40
92/92 [==============================] - 0s 4ms/step - loss: 0.0748 - accuracy: 0.97
11 - val_loss: 0.2056 - val_accuracy: 0.9313
Epoch 7/40
92/92 [==============================] - 0s 4ms/step - loss: 0.0765 - accuracy: 0.97
16 - val_loss: 0.1482 - val_accuracy: 0.9443
Epoch 8/40
92/92 [==============================] - 0s 4ms/step - loss: 0.0573 - accuracy: 0.97
99 - val_loss: 0.1537 - val_accuracy: 0.9477
Epoch 9/40
92/92 [==============================] - 0s 4ms/step - loss: 0.0579 - accuracy: 0.97
82 - val_loss: 0.1341 - val_accuracy: 0.9456
Epoch 10/40
92/92 [==============================] - 0s 4ms/step - loss: 0.0485 - accuracy: 0.98
21 - val_loss: 0.1604 - val_accuracy: 0.9422
Epoch 11/40
92/92 [==============================] - 0s 4ms/step - loss: 0.0474 - accuracy: 0.98
16 - val_loss: 0.1757 - val_accuracy: 0.9415
Epoch 12/40
92/92 [==============================] - 0s 4ms/step - loss: 0.0487 - accuracy: 0.98
15 - val_loss: 0.1990 - val_accuracy: 0.9327
Epoch 13/40
92/92 [==============================] - 0s 4ms/step - loss: 0.0556 - accuracy: 0.97
60 - val_loss: 0.1575 - val_accuracy: 0.9483
Epoch 14/40
92/92 [==============================] - 0s 3ms/step - loss: 0.0435 - accuracy: 0.98
37 - val_loss: 0.1284 - val_accuracy: 0.9524
Epoch 15/40
92/92 [==============================] - 0s 5ms/step - loss: 0.0555 - accuracy: 0.97
70 - val_loss: 0.1285 - val_accuracy: 0.9497
Epoch 16/40
92/92 [==============================] - 0s 4ms/step - loss: 0.0342 - accuracy: 0.98
61 - val_loss: 0.1308 - val_accuracy: 0.9517
Epoch 17/40
92/92 [==============================] - 0s 5ms/step - loss: 0.0447 - accuracy: 0.98
04 - val_loss: 0.1505 - val_accuracy: 0.9517
Epoch 18/40
92/92 [==============================] - 0s 5ms/step - loss: 0.0549 - accuracy: 0.97
91 - val_loss: 0.1501 - val_accuracy: 0.9517
Epoch 19/40
```

```
92/92 [==============================] - 0s 5ms/step - loss: 0.0714 - accuracy: 0.97
48 - val_loss: 0.1655 - val_accuracy: 0.9422
```

Training Curves

In [7]:
```
# TODO:
# Plot:
# 1. training accuracy vs validation accuracy
# 2. training loss vs validation loss

# <-- Enter your code here <--#
plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Baseline Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Baseline Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```

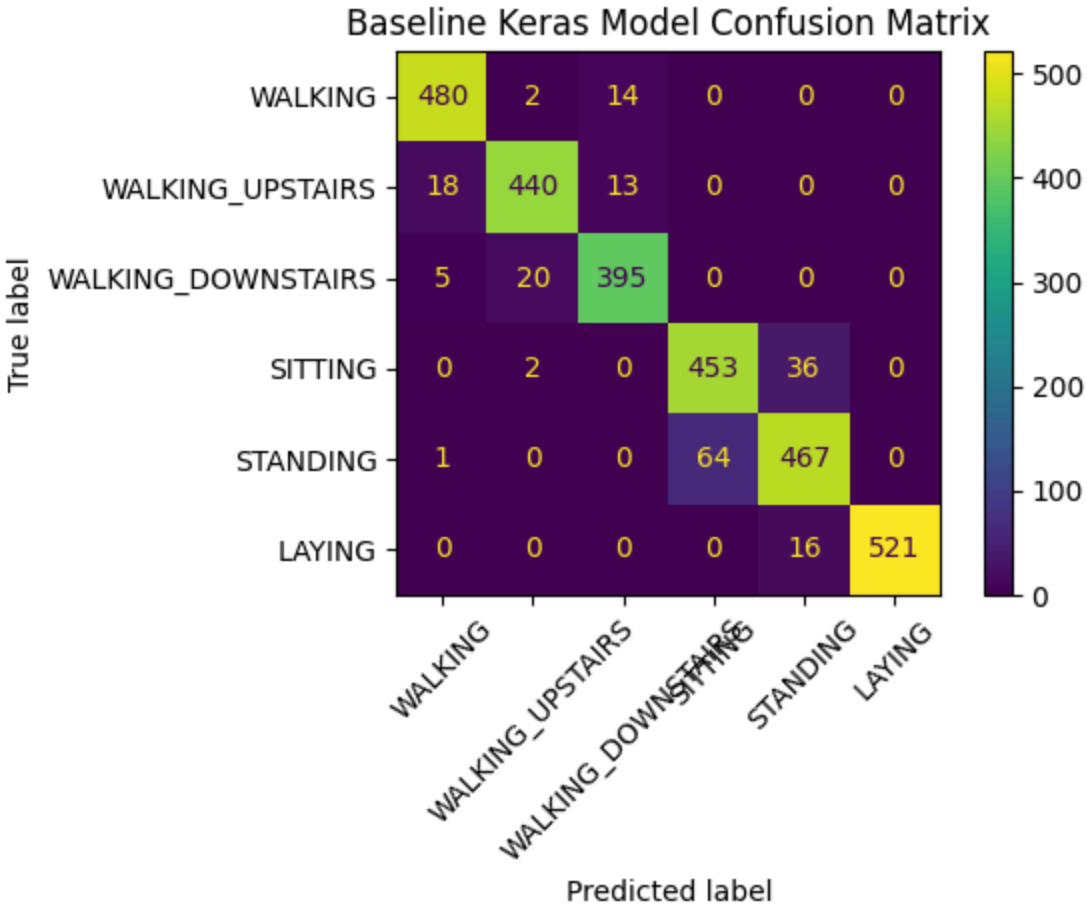




6. Evaluate the Baseline Keras Model

In [8]:
```
# TODO:
# 1. Predict class probabilities on X_test
# 2. Convert probabilities to class labels using argmax
# 3. Compute the test accuracy
# 4. Print the classification report
# 5. Plot the confusion matrix

# <-- Enter your code here <--#
baseline_probs = baseline_model.predict(X_test, verbose=0)
baseline_pred = np.argmax(baseline_probs, axis=1)
baseline_keras_acc = accuracy_score(y_test, baseline_pred)

print(f"Baseline Keras test accuracy: {baseline_keras_acc:.4f}")
print(classification_report(y_test, baseline_pred, target_names=class_names))

cm = confusion_matrix(y_test, baseline_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
disp.plot(xticks_rotation=45, values_format="d")
plt.title("Baseline Keras Model Confusion Matrix")
plt.tight_layout()
plt.show()
```

```
Baseline Keras test accuracy: 0.9352
                    precision    recall  f1-score   support

           WALKING       0.95      0.97      0.96       496
  WALKING_UPSTAIRS       0.95      0.93      0.94       471
WALKING_DOWNSTAIRS       0.94      0.94      0.94       420
           SITTING       0.88      0.92      0.90       491
          STANDING       0.90      0.88      0.89       532
            LAYING       1.00      0.97      0.98       537

          accuracy                           0.94      2947
         macro avg       0.94      0.94      0.94      2947
      weighted avg       0.94      0.94      0.94      2947
```



Part I: Model Pruning with Sparsity

In this part, we apply **magnitude-based pruning** to the DNN. The key idea is to gradually set small-magnitude weights to zero during training.

We will compare:

1. the baseline TensorFlow Lite model,
2. the pruned model converted **without** stripping the pruning wrappers, and
3. the stripped sparse model converted with **experimental sparsity-aware optimization**.

7. TensorFlow Lite Utilities

The following helper functions are used to:

- convert Keras models to TensorFlow Lite,
- evaluate TensorFlow Lite models on the test set, and
- measure model size.

In [9]:
```
def save_binary_model(model_content, filename):
    with open(filename, "wb") as f:
        f.write(model_content)
    return os.path.getsize(filename) / 1024.0  # KB

def evaluate_tflite_model(tflite_model, X, y_true):
    interpreter = tf.lite.Interpreter(model_content=tflite_model)
    interpreter.allocate_tensors()

    input_details = interpreter.get_input_details()[0]
    output_details = interpreter.get_output_details()[0]

    input_scale, input_zero_point = input_details["quantization"]
    output_scale, output_zero_point = output_details["quantization"]

    y_pred = []

    for i in range(len(X)):
        x = X[i:i+1].astype(np.float32)

        # TODO:
        # Quantize the input only when the TFLite input tensor expects int8 or uint
        # Otherwise keep the input in the required floating-point dtype.

        # <-- Enter your code here <--#
        # Quantize input only for integer input tensors.
        if input_details["dtype"] in (np.int8, np.uint8):
            x = x / input_scale + input_zero_point
            x = np.round(x)
            x = np.clip(x, np.iinfo(input_details["dtype"]).min, np.iinfo(input_det
            x = x.astype(input_details["dtype"])
        else:
            x = x.astype(input_details["dtype"])

        interpreter.set_tensor(input_details["index"], x)
        interpreter.invoke()

        output = interpreter.get_tensor(output_details["index"])

        # TODO:
        # Dequantize the output only when the output tensor is int8 or uint8.

        # <-- Enter your code here <--#
        # Dequantize output only for integer output tensors.
```

```
        if output_details["dtype"] in (np.int8, np.uint8):
            output = (output.astype(np.float32) - output_zero_point) * output_scale

        pred = int(np.argmax(output, axis=1)[0])
        y_pred.append(pred)

    acc = accuracy_score(y_true, y_pred)
    return acc, np.array(y_pred)

def convert_to_tflite_fp32(model):
    # TODO:
    # Create a TFLiteConverter from the Keras model and return the converted FP32 m

    # <-- Enter your code here <--#
    converter = tf.lite.TFLiteConverter.from_keras_model(model)
    return converter.convert()
```

8. Convert the Baseline Model to TensorFlow Lite

In [10]:

```
# TODO:
# Convert the baseline model to FP32 TensorFlow Lite.
# Save the .tflite file, compute its size in KB, and evaluate it on X_test.

# <-- Enter your code here <--#
baseline_tflite_fp32 = convert_to_tflite_fp32(baseline_model)
baseline_tflite_fp32_size_kb = save_binary_model(
    baseline_tflite_fp32,
    "baseline_fp32.tflite"
)
baseline_tflite_fp32_acc, baseline_tflite_fp32_pred = evaluate_tflite_model(
    baseline_tflite_fp32,
    X_test,
    y_test
)

print(f"Baseline FP32 TFLite size: {baseline_tflite_fp32_size_kb:.2f} KB")
print(f"Baseline FP32 TFLite test accuracy: {baseline_tflite_fp32_acc:.4f}")
```

```
INFO:tensorflow:Assets written to: /tmp/tmp6eq95wmt/assets
```

```
INFO:tensorflow:Assets written to: /tmp/tmp6eq95wmt/assets
Baseline FP32 TFLite size: 726.72 KB
Baseline FP32 TFLite test accuracy: 0.9352
```

```
2026-04-23 21:57:10.956040: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:378] Ignored output_format.
2026-04-23 21:57:10.956109: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:381] Ignored drop_control_dependency.
2026-04-23 21:57:10.957240: I tensorflow/cc/saved_model/reader.cc:83] Reading SavedM
odel from: /tmp/tmp6eq95wmt
2026-04-23 21:57:10.958397: I tensorflow/cc/saved_model/reader.cc:51] Reading meta g
raph with tags { serve }
2026-04-23 21:57:10.958417: I tensorflow/cc/saved_model/reader.cc:146] Reading Saved
Model debug info (if present) from: /tmp/tmp6eq95wmt
2026-04-23 21:57:10.964244: I tensorflow/compiler/mlir/mlir_graph_optimization_pass.
cc:382] MLIR V1 optimization pass is not enabled
2026-04-23 21:57:10.965246: I tensorflow/cc/saved_model/loader.cc:233] Restoring Sav
edModel bundle.
2026-04-23 21:57:11.029003: I tensorflow/cc/saved_model/loader.cc:217] Running initi
alization op on SavedModel bundle at path: /tmp/tmp6eq95wmt
2026-04-23 21:57:11.042746: I tensorflow/cc/saved_model/loader.cc:316] SavedModel lo
ad for tags { serve }; Status: success: OK. Took 85446 microseconds.
2026-04-23 21:57:11.084741: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_ut
il.cc:269] disabling MLIR crash reproducer, set env var `MLIR_CRASH_REPRODUCER_DIREC
TORY` to enable.
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
```

9. Apply Magnitude-Based Pruning

We will prune the DNN using a **polynomial decay schedule**:

- start from low sparsity,
- gradually increase sparsity during training, and
- finish with a highly sparse model.

After training, we will compare:

- the pruned model **with** the pruning wrappers still present, and
- the final sparse model after applying `strip_pruning(...)`.

In [11]:
```
from tensorflow_model_optimization.sparsity.keras import (
    prune_low_magnitude,
    PolynomialDecay,
    UpdatePruningStep,
    strip_pruning
)

pruning_epochs = 12
batch_size = 64

# TODO:
# Compute steps_per_epoch using 80% of the training set and the selected batch size
# Define pruning_params using PolynomialDecay with:
# - initial_sparsity=0.20
# - final_sparsity=0.85
# - begin_step=0
# - end_step=steps_per_epoch * pruning_epochs
```

```
# <-- Enter your code here <--#
steps_per_epoch = math.ceil((X_train.shape[0] * 0.8) / batch_size)
end_step = steps_per_epoch * pruning_epochs

pruning_params = {
    "pruning_schedule": PolynomialDecay(
        initial_sparsity=0.20,
        final_sparsity=0.85,
        begin_step=0,
        end_step=end_step
    )
}

# TODO:
# Create the pruned model by wrapping a fresh baseline DNN with prune_low_magnitude
# Compile it with Adam(1e-3), sparse_categorical_crossentropy, and accuracy.

# <-- Enter your code here <--#
fresh_model_for_pruning = build_baseline_model(num_features, num_classes)
pruned_model = prune_low_magnitude(fresh_model_for_pruning, **pruning_params)

pruned_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

pruning_callbacks = [
    UpdatePruningStep(),
    keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        patience=3,
        restore_best_weights=True
    )
]

# TODO:
# Train the pruned model using:
# - validation_split=0.2
# - epochs=pruning_epochs
# - batch_size=batch_size
# - callbacks=pruning_callbacks

# <-- Enter your code here <--#
pruned_history = pruned_model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=pruning_epochs,
    batch_size=batch_size,
    callbacks=pruning_callbacks,
    verbose=1
)
```

```
Epoch 1/12
92/92 [==============================] - 3s 8ms/step - loss: 0.4751 - accuracy: 0.80
67 - val_loss: 0.2226 - val_accuracy: 0.9218
Epoch 2/12
92/92 [==============================] - 0s 5ms/step - loss: 0.1663 - accuracy: 0.93
67 - val_loss: 0.1592 - val_accuracy: 0.9293
Epoch 3/12
92/92 [==============================] - 1s 6ms/step - loss: 0.1311 - accuracy: 0.94
98 - val_loss: 0.2181 - val_accuracy: 0.9279
Epoch 4/12
92/92 [==============================] - 1s 6ms/step - loss: 0.0919 - accuracy: 0.96
91 - val_loss: 0.1689 - val_accuracy: 0.9320
Epoch 5/12
92/92 [==============================] - 1s 6ms/step - loss: 0.1259 - accuracy: 0.95
15 - val_loss: 0.1484 - val_accuracy: 0.9395
Epoch 6/12
92/92 [==============================] - 0s 5ms/step - loss: 0.1224 - accuracy: 0.95
77 - val_loss: 0.1762 - val_accuracy: 0.9354
Epoch 7/12
92/92 [==============================] - 1s 7ms/step - loss: 0.2871 - accuracy: 0.92
94 - val_loss: 0.1576 - val_accuracy: 0.9388
Epoch 8/12
92/92 [==============================] - 1s 6ms/step - loss: 0.2122 - accuracy: 0.93
42 - val_loss: 0.2207 - val_accuracy: 0.9205
```

10. Convert the Pruned Model Before and After Stripping the Pruning Wrappers

First, we convert the pruned model **with** the pruning wrappers still attached.

Next, we strip the pruning wrappers and convert the resulting sparse model with:

- `tf.lite.Optimize.EXPERIMENTAL_SPARSITY`

This is the proper way to preserve sparsity in the exported TensorFlow Lite model.

In [12]:
```
# TODO:
# 1. Convert the pruned model WITH the pruning wrappers still attached to FP32 Tens
# 2. Save the model and evaluate it on X_test.
# 3. Strip the pruning wrappers using strip_pruning(...).
# 4. Convert the stripped model with tf.lite.Optimize.EXPERIMENTAL_SPARSITY.
# 5. Save the stripped sparse model and evaluate it on X_test.

# <-- Enter your code here <--#
# 1-2. Convert/evaluate the pruned model with pruning wrappers still attached.
pruned_tflite_fp32_with_mask = convert_to_tflite_fp32(pruned_model)
pruned_tflite_fp32_with_mask_size_kb = save_binary_model(
    pruned_tflite_fp32_with_mask,
    "pruned_fp32_with_mask.tflite"
)
pruned_tflite_fp32_with_mask_acc, pruned_tflite_fp32_with_mask_pred = evaluate_tfli
    pruned_tflite_fp32_with_mask,
    X_test,
```

```
    y_test
)

# 3. Strip pruning wrappers to finalize the sparse model.
stripped_pruned_model = strip_pruning(pruned_model)

# 4-5. Convert/evaluate the stripped model with sparsity-aware TFLite optimization.
converter = tf.lite.TFLiteConverter.from_keras_model(stripped_pruned_model)
converter.optimizations = [tf.lite.Optimize.EXPERIMENTAL_SPARSITY]
stripped_sparse_tflite_fp32 = converter.convert()

stripped_sparse_tflite_fp32_size_kb = save_binary_model(
    stripped_sparse_tflite_fp32,
    "stripped_sparse_fp32.tflite"
)
stripped_sparse_tflite_fp32_acc, stripped_sparse_tflite_fp32_pred = evaluate_tflite
    stripped_sparse_tflite_fp32,
    X_test,
    y_test
)

print(f"Pruned FP32 with mask size: {pruned_tflite_fp32_with_mask_size_kb:.2f} KB")
print(f"Pruned FP32 with mask accuracy: {pruned_tflite_fp32_with_mask_acc:.4f}")
print(f"Stripped sparse FP32 size: {stripped_sparse_tflite_fp32_size_kb:.2f} KB")
print(f"Stripped sparse FP32 accuracy: {stripped_sparse_tflite_fp32_acc:.4f}")
```

```
INFO:tensorflow:Assets written to: /tmp/tmp4jtl55ao/assets
```

```
INFO:tensorflow:Assets written to: /tmp/tmp4jtl55ao/assets
2026-04-23 21:57:20.774116: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:378] Ignored output_format.
2026-04-23 21:57:20.774181: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:381] Ignored drop_control_dependency.
2026-04-23 21:57:20.774354: I tensorflow/cc/saved_model/reader.cc:83] Reading SavedM
odel from: /tmp/tmp4jtl55ao
2026-04-23 21:57:20.779878: I tensorflow/cc/saved_model/reader.cc:51] Reading meta g
raph with tags { serve }
2026-04-23 21:57:20.779922: I tensorflow/cc/saved_model/reader.cc:146] Reading Saved
Model debug info (if present) from: /tmp/tmp4jtl55ao
2026-04-23 21:57:20.795142: I tensorflow/cc/saved_model/loader.cc:233] Restoring Sav
edModel bundle.
2026-04-23 21:57:20.863399: I tensorflow/cc/saved_model/loader.cc:217] Running initi
alization op on SavedModel bundle at path: /tmp/tmp4jtl55ao
2026-04-23 21:57:20.884303: I tensorflow/cc/saved_model/loader.cc:316] SavedModel lo
ad for tags { serve }; Status: success: OK. Took 109946 microseconds.
```

```
INFO:tensorflow:Assets written to: /tmp/tmp0tdws3ep/assets
```

```
INFO:tensorflow:Assets written to: /tmp/tmp0tdws3ep/assets
Pruned FP32 with mask size: 1454.18 KB
Pruned FP32 with mask accuracy: 0.9192
Stripped sparse FP32 size: 338.85 KB
Stripped sparse FP32 accuracy: 0.9192
```

```
2026-04-23 21:57:22.305496: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:378] Ignored output_format.
2026-04-23 21:57:22.305572: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:381] Ignored drop_control_dependency.
2026-04-23 21:57:22.305767: I tensorflow/cc/saved_model/reader.cc:83] Reading SavedM
odel from: /tmp/tmp0tdws3ep
2026-04-23 21:57:22.306544: I tensorflow/cc/saved_model/reader.cc:51] Reading meta g
raph with tags { serve }
2026-04-23 21:57:22.306561: I tensorflow/cc/saved_model/reader.cc:146] Reading Saved
Model debug info (if present) from: /tmp/tmp0tdws3ep
2026-04-23 21:57:22.308380: I tensorflow/cc/saved_model/loader.cc:233] Restoring Sav
edModel bundle.
2026-04-23 21:57:22.327788: I tensorflow/cc/saved_model/loader.cc:217] Running initi
alization op on SavedModel bundle at path: /tmp/tmp0tdws3ep
2026-04-23 21:57:22.334261: I tensorflow/cc/saved_model/loader.cc:316] SavedModel lo
ad for tags { serve }; Status: success: OK. Took 28494 microseconds.
```

11. Part I Comparison: Accuracy and Model Size

In [13]:

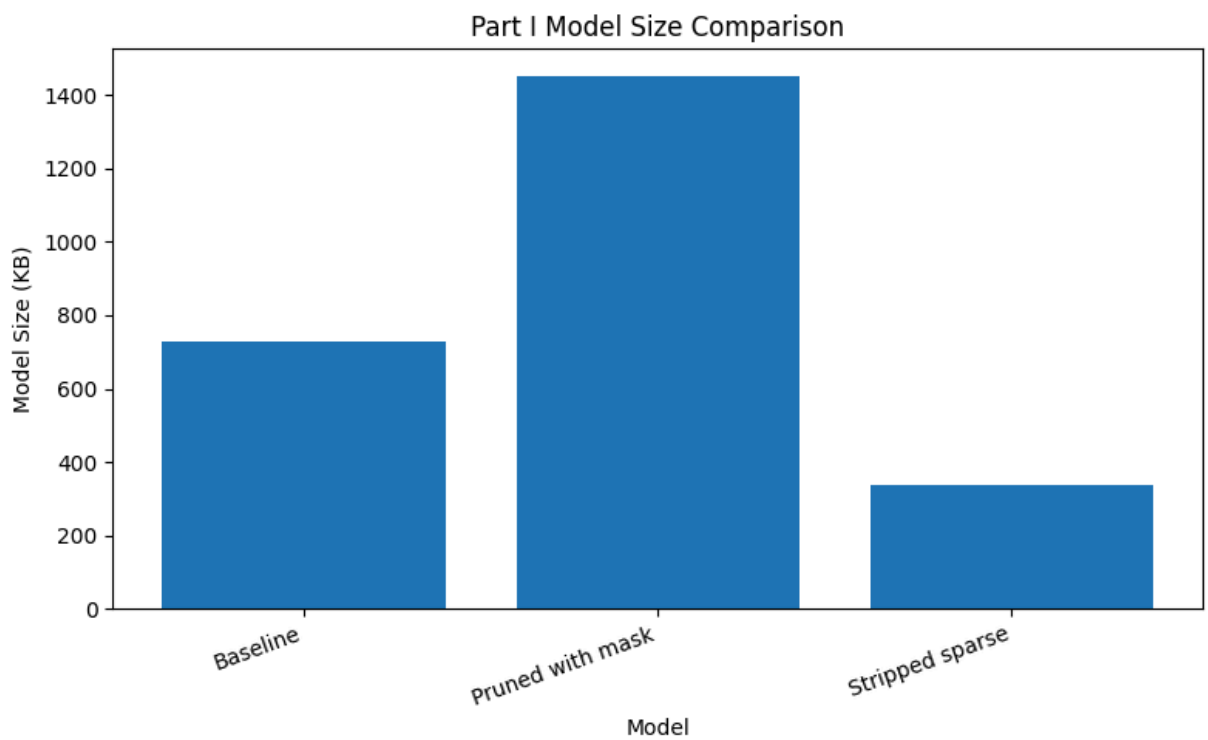
```python
# TODO:
# Create a comparison DataFrame for Part I with the columns:
# Model, Format, Test Accuracy, Model Size (KB)

# Include:
# - baseline FP32 TFLite
# - pruned FP32 TFLite with mask
# - stripped sparse FP32 TFLite

# <-- Enter your code here <--#
part_i_results = pd.DataFrame([
    {
        "Model": "Baseline",
        "Format": "FP32 TFLite",
        "Test Accuracy": baseline_tflite_fp32_acc,
        "Model Size (KB)": baseline_tflite_fp32_size_kb
    },
    {
        "Model": "Pruned with mask",
        "Format": "FP32 TFLite",
        "Test Accuracy": pruned_tflite_fp32_with_mask_acc,
        "Model Size (KB)": pruned_tflite_fp32_with_mask_size_kb
    },
    {
        "Model": "Stripped sparse",
        "Format": "Sparse FP32 TFLite",
        "Test Accuracy": stripped_sparse_tflite_fp32_acc,
        "Model Size (KB)": stripped_sparse_tflite_fp32_size_kb
    }
])

part_i_results
```

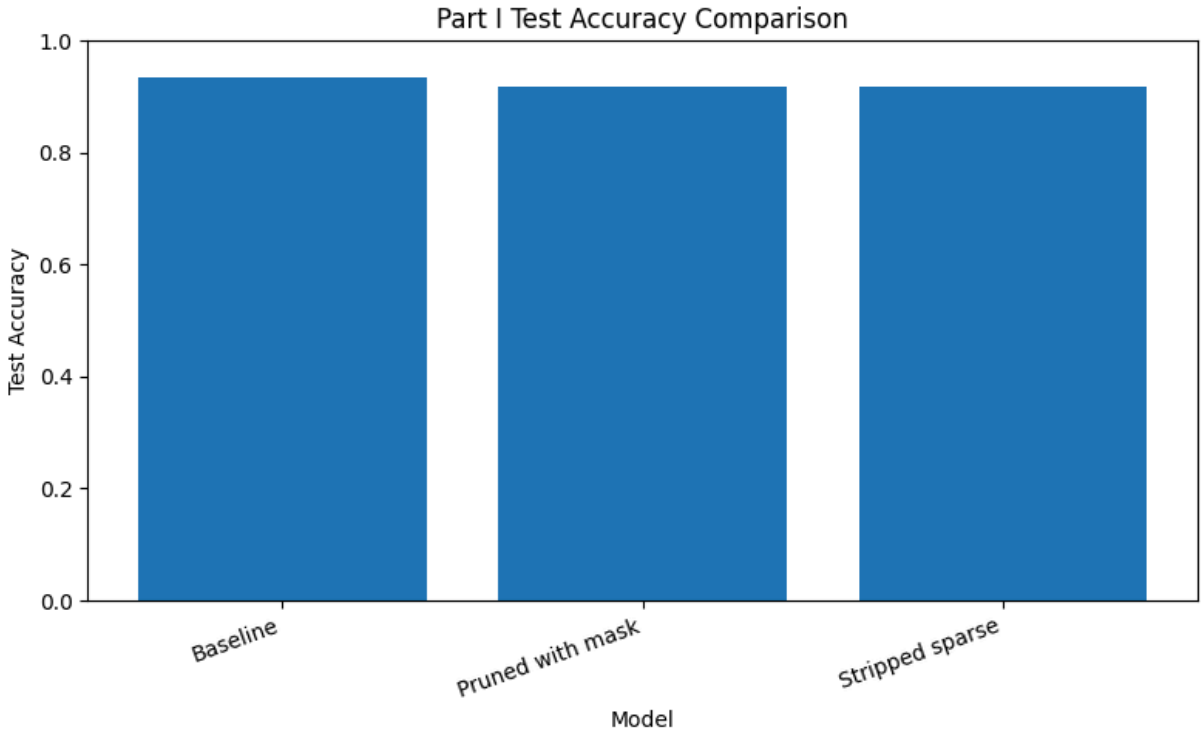
Out[13]:

	Model	Format	Test Accuracy	Model Size (KB)
0	Baseline	FP32 TFLite	0.935188	726.718750
1	Pruned with mask	FP32 TFLite	0.919240	1454.175781
2	Stripped sparse	Sparse FP32 TFLite	0.919240	338.851562

In [14]:

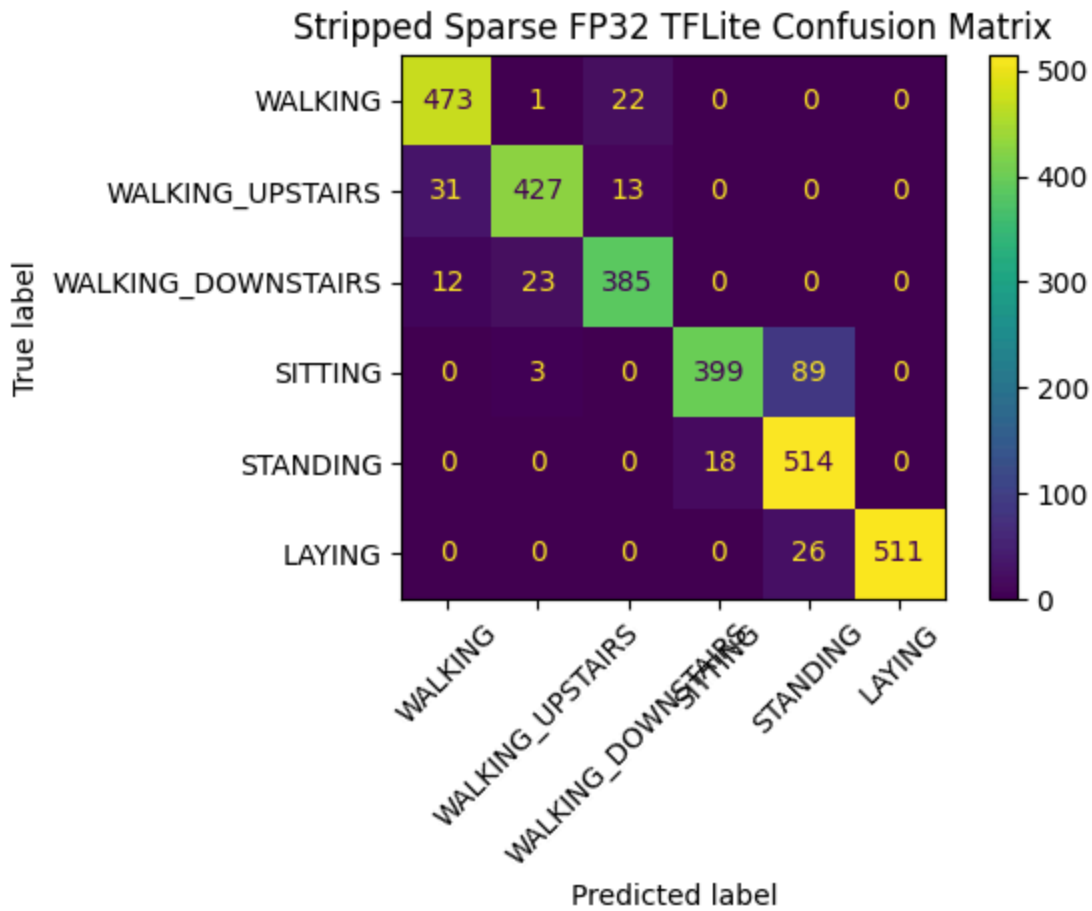
```
# TODO:
# Plot:
# 1. a bar chart of the Part I model sizes
# 2. a bar chart of the Part I test accuracies

# <-- Enter your code here <--#
plt.figure(figsize=(8, 5))
plt.bar(part_i_results["Model"], part_i_results["Model Size (KB)"])
plt.title("Part I Model Size Comparison")
plt.xlabel("Model")
plt.ylabel("Model Size (KB)")
plt.xticks(rotation=20, ha="right")
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
plt.bar(part_i_results["Model"], part_i_results["Test Accuracy"])
plt.title("Part I Test Accuracy Comparison")
plt.xlabel("Model")
plt.ylabel("Test Accuracy")
plt.ylim(0, 1.0)
plt.xticks(rotation=20, ha="right")
plt.tight_layout()
plt.show()
```





Confusion Matrix for the Stripped Sparse Model

In [15]:
```
# TODO:
# Plot the confusion matrix for the stripped sparse TFLite model.
# Print the classification report for the stripped sparse TFLite model.

# <-- Enter your code here <--#
cm = confusion_matrix(y_test, stripped_sparse_tflite_fp32_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
disp.plot(xticks_rotation=45, values_format="d")
plt.title("Stripped Sparse FP32 TFLite Confusion Matrix")
plt.tight_layout()
plt.show()

print(classification_report(y_test, stripped_sparse_tflite_fp32_pred, target_names=
```




```
                    precision    recall  f1-score   support

           WALKING       0.92      0.95      0.93       496
  WALKING_UPSTAIRS       0.94      0.91      0.92       471
WALKING_DOWNSTAIRS       0.92      0.92      0.92       420
           SITTING       0.96      0.81      0.88       491
          STANDING       0.82      0.97      0.89       532
            LAYING       1.00      0.95      0.98       537

          accuracy                           0.92      2947
         macro avg       0.92      0.92      0.92      2947
      weighted avg       0.92      0.92      0.92      2947
```

Part II: Model Pruning + Float16 Quantization

In this part, we combine **pruning** and **float16 quantization**.

We will compare:

1. the pruned TensorFlow Lite model **with** the pruning wrappers still attached, after float16 quantization, and

2. the stripped sparse TensorFlow Lite model after **both** sparsity-aware optimization and float16 quantization.

This lets us observe whether properly finalizing the pruned model leads to a more compact deployable representation.

In [16]:
```
# TODO:
# Part II: combine pruning and float16 quantization.
#
# 1. Convert the pruned model with mask using:
#    - optimizations = [tf.lite.Optimize.DEFAULT]
#    - supported_types = [tf.float16]
# 2. Save and evaluate the float16 model with mask.
# 3. Convert the stripped sparse model using:
#    - optimizations = [tf.lite.Optimize.DEFAULT, tf.lite.Optimize.EXPERIMENTAL_SPA
#    - supported_types = [tf.float16]
# 4. Save and evaluate the stripped sparse + float16 model.

# <-- Enter your code here <--#
# 1-2. Convert/evaluate the pruned model with mask using float16 quantization.
converter = tf.lite.TFLiteConverter.from_keras_model(pruned_model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_types = [tf.float16]
pruned_tflite_float16_with_mask = converter.convert()

pruned_tflite_float16_with_mask_size_kb = save_binary_model(
    pruned_tflite_float16_with_mask,
    "pruned_float16_with_mask.tflite"
)
pruned_tflite_float16_with_mask_acc, pruned_tflite_float16_with_mask_pred = evaluat
    pruned_tflite_float16_with_mask,
    X_test,
    y_test
)

# 3-4. Convert/evaluate the stripped sparse model using sparsity-aware float16 quan
converter = tf.lite.TFLiteConverter.from_keras_model(stripped_pruned_model)
converter.optimizations = [tf.lite.Optimize.DEFAULT, tf.lite.Optimize.EXPERIMENTAL_
converter.target_spec.supported_types = [tf.float16]
stripped_sparse_tflite_float16 = converter.convert()

stripped_sparse_tflite_float16_size_kb = save_binary_model(
    stripped_sparse_tflite_float16,
    "stripped_sparse_float16.tflite"
)
stripped_sparse_tflite_float16_acc, stripped_sparse_tflite_float16_pred = evaluate_
    stripped_sparse_tflite_float16,
    X_test,
    y_test
)

print(f"Pruned float16 with mask size: {pruned_tflite_float16_with_mask_size_kb:.2f
print(f"Pruned float16 with mask accuracy: {pruned_tflite_float16_with_mask_acc:.4f
```

```
print(f"Stripped sparse float16 size: {stripped_sparse_tflite_float16_size_kb:.2f}
print(f"Stripped sparse float16 accuracy: {stripped_sparse_tflite_float16_acc:.4f}"
```

```
INFO:tensorflow:Assets written to: /tmp/tmpuod2z_ru/assets
```

```
INFO:tensorflow:Assets written to: /tmp/tmpuod2z_ru/assets
2026-04-23 21:57:25.408705: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:378] Ignored output_format.
2026-04-23 21:57:25.408775: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:381] Ignored drop_control_dependency.
2026-04-23 21:57:25.408953: I tensorflow/cc/saved_model/reader.cc:83] Reading SavedM
odel from: /tmp/tmpuod2z_ru
2026-04-23 21:57:25.412388: I tensorflow/cc/saved_model/reader.cc:51] Reading meta g
raph with tags { serve }
2026-04-23 21:57:25.412416: I tensorflow/cc/saved_model/reader.cc:146] Reading Saved
Model debug info (if present) from: /tmp/tmpuod2z_ru
2026-04-23 21:57:25.423007: I tensorflow/cc/saved_model/loader.cc:233] Restoring Sav
edModel bundle.
2026-04-23 21:57:25.485021: I tensorflow/cc/saved_model/loader.cc:217] Running initi
alization op on SavedModel bundle at path: /tmp/tmpuod2z_ru
2026-04-23 21:57:25.509042: I tensorflow/cc/saved_model/loader.cc:316] SavedModel lo
ad for tags { serve }; Status: success: OK. Took 100089 microseconds.
```

```
INFO:tensorflow:Assets written to: /tmp/tmppu88mcha/assets
```

```
INFO:tensorflow:Assets written to: /tmp/tmppu88mcha/assets
2026-04-23 21:57:26.928120: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:378] Ignored output_format.
2026-04-23 21:57:26.928200: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:381] Ignored drop_control_dependency.
```

```
Pruned float16 with mask size: 732.59 KB
Pruned float16 with mask accuracy: 0.9192
Stripped sparse float16 size: 223.71 KB
Stripped sparse float16 accuracy: 0.9192
```

```
2026-04-23 21:57:26.928438: I tensorflow/cc/saved_model/reader.cc:83] Reading SavedM
odel from: /tmp/tmppu88mcha
2026-04-23 21:57:26.929257: I tensorflow/cc/saved_model/reader.cc:51] Reading meta g
raph with tags { serve }
2026-04-23 21:57:26.929275: I tensorflow/cc/saved_model/reader.cc:146] Reading Saved
Model debug info (if present) from: /tmp/tmppu88mcha
2026-04-23 21:57:26.931074: I tensorflow/cc/saved_model/loader.cc:233] Restoring Sav
edModel bundle.
2026-04-23 21:57:26.949968: I tensorflow/cc/saved_model/loader.cc:217] Running initi
alization op on SavedModel bundle at path: /tmp/tmppu88mcha
2026-04-23 21:57:26.957611: I tensorflow/cc/saved_model/loader.cc:316] SavedModel lo
ad for tags { serve }; Status: success: OK. Took 29174 microseconds.
```

12. Part II Comparison: Accuracy and Model Size

In [17]:

```
# TODO:
# Create a Part II comparison DataFrame with the columns:
# Model, Format, Test Accuracy, Model Size (KB)
#
# Include:
# - pruned FP32 with mask
# - stripped sparse FP32
# - pruned float16 with mask
```

```
# - stripped sparse float16

# <-- Enter your code here <--#
part_ii_results = pd.DataFrame([
    {
        "Model": "Pruned with mask",
        "Format": "FP32 TFLite",
        "Test Accuracy": pruned_tflite_fp32_with_mask_acc,
        "Model Size (KB)": pruned_tflite_fp32_with_mask_size_kb
    },
    {
        "Model": "Stripped sparse",
        "Format": "Sparse FP32 TFLite",
        "Test Accuracy": stripped_sparse_tflite_fp32_acc,
        "Model Size (KB)": stripped_sparse_tflite_fp32_size_kb
    },
    {
        "Model": "Pruned with mask",
        "Format": "Float16 TFLite",
        "Test Accuracy": pruned_tflite_float16_with_mask_acc,
        "Model Size (KB)": pruned_tflite_float16_with_mask_size_kb
    },
    {
        "Model": "Stripped sparse",
        "Format": "Sparse Float16 TFLite",
        "Test Accuracy": stripped_sparse_tflite_float16_acc,
        "Model Size (KB)": stripped_sparse_tflite_float16_size_kb
    }
])

part_ii_results
```

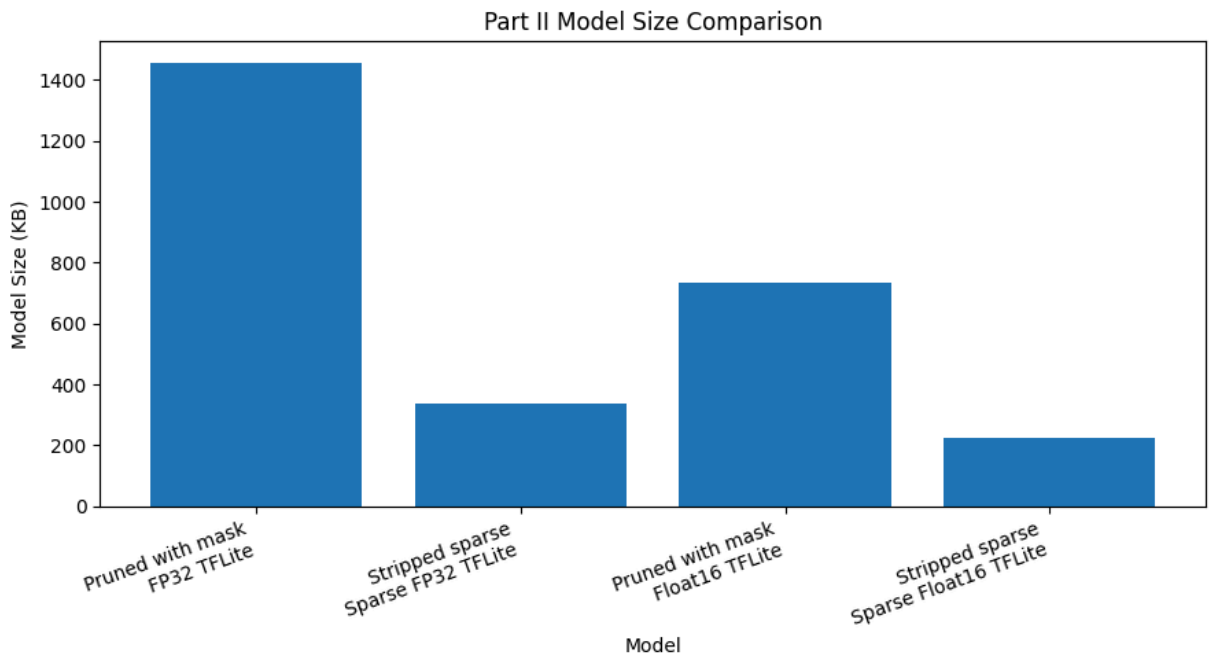
Out[17]:

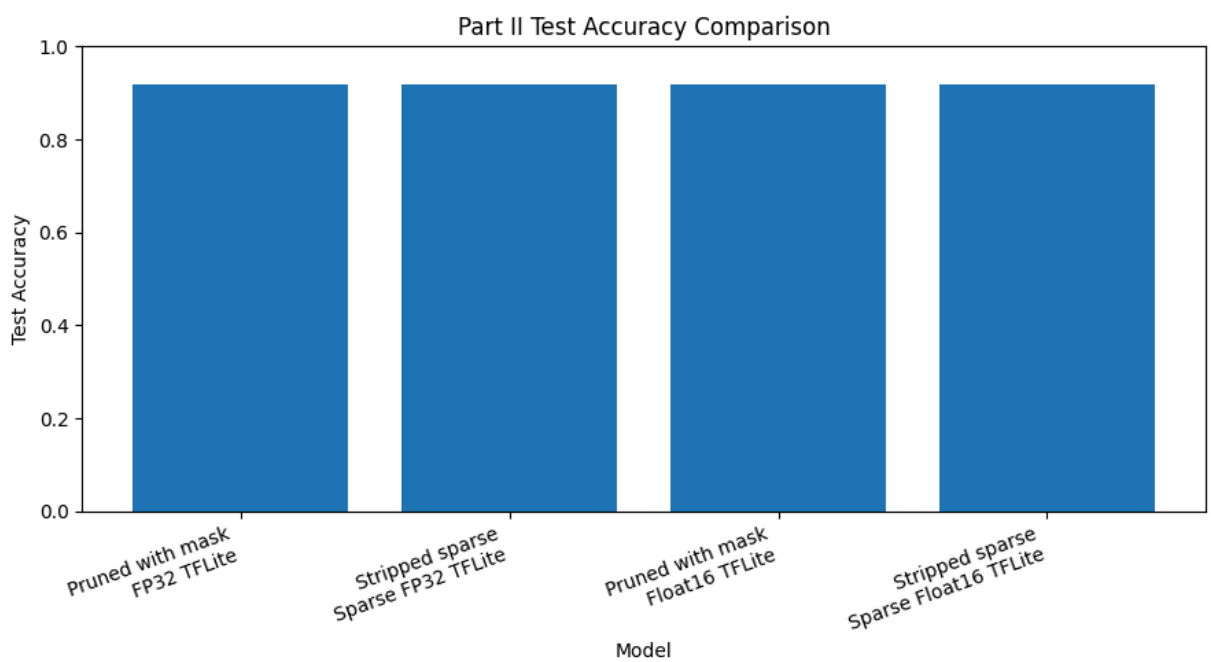
	Model	Format	Test Accuracy	Model Size (KB)
0	Pruned with mask	FP32 TFLite	0.91924	1454.175781
1	Stripped sparse	Sparse FP32 TFLite	0.91924	338.851562
2	Pruned with mask	Float16 TFLite	0.91924	732.585938
3	Stripped sparse	Sparse Float16 TFLite	0.91924	223.710938

In [18]:

```
# TODO:
# Plot:
# 1. a bar chart of Part II model sizes
# 2. a bar chart of Part II test accuracies

# <-- Enter your code here <--#
labels = part_ii_results["Model"] + "\n" + part_ii_results["Format"]

plt.figure(figsize=(9, 5))
plt.bar(labels, part_ii_results["Model Size (KB)"])
plt.title("Part II Model Size Comparison")
plt.xlabel("Model")
plt.ylabel("Model Size (KB)")
plt.xticks(rotation=20, ha="right")
```

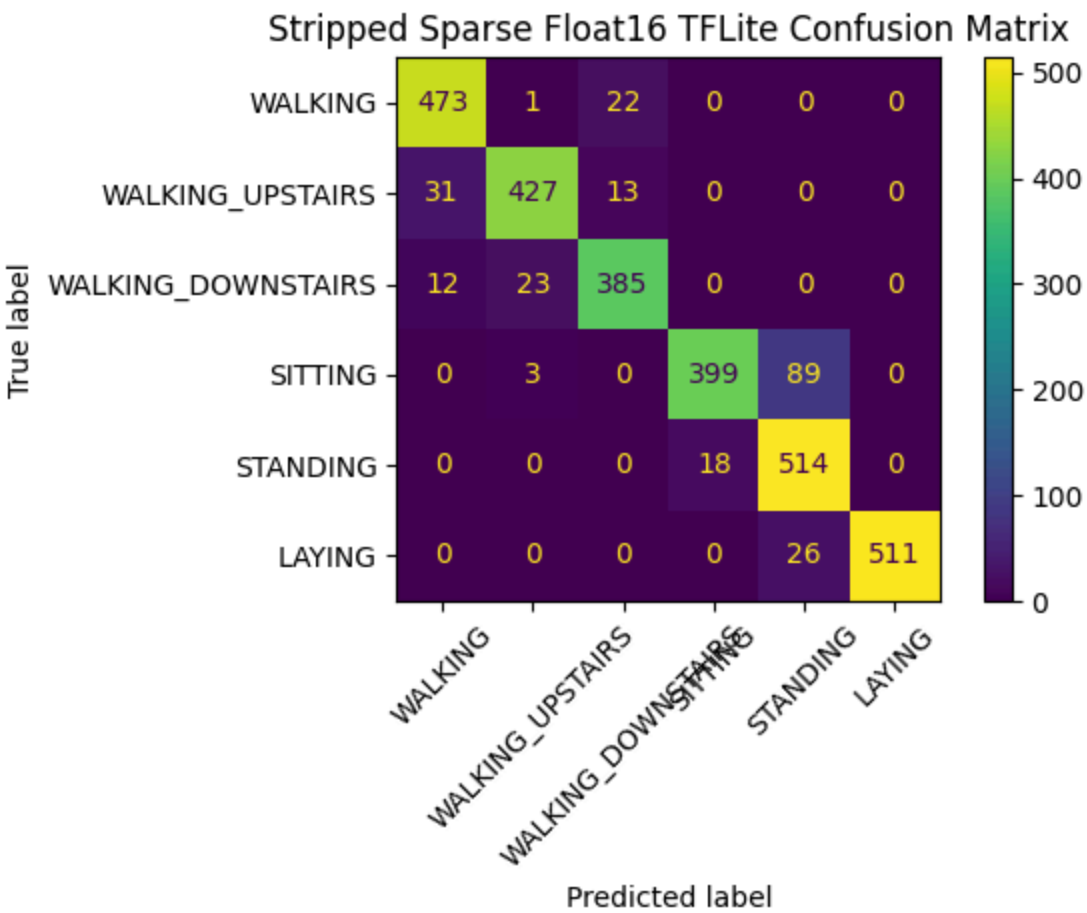
```
plt.tight_layout()
plt.show()

plt.figure(figsize=(9, 5))
plt.bar(labels, part_ii_results["Test Accuracy"])
plt.title("Part II Test Accuracy Comparison")
plt.xlabel("Model")
plt.ylabel("Test Accuracy")
plt.ylim(0, 1.0)
plt.xticks(rotation=20, ha="right")
plt.tight_layout()
plt.show()
```





Confusion Matrix for the Stripped Sparse + Float16 Model

In [19]:
```
# TODO:
# Plot the confusion matrix for the stripped sparse + float16 TFLite model.
# Print the classification report for the stripped sparse + float16 TFLite model.

# <-- Enter your code here <--#
cm = confusion_matrix(y_test, stripped_sparse_tflite_float16_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
disp.plot(xticks_rotation=45, values_format="d")
plt.title("Stripped Sparse Float16 TFLite Confusion Matrix")
plt.tight_layout()
plt.show()

print(classification_report(y_test, stripped_sparse_tflite_float16_pred, target_nam
```




```
                    precision    recall  f1-score   support

           WALKING       0.92      0.95      0.93       496
  WALKING_UPSTAIRS       0.94      0.91      0.92       471
WALKING_DOWNSTAIRS       0.92      0.92      0.92       420
           SITTING       0.96      0.81      0.88       491
          STANDING       0.82      0.97      0.89       532
            LAYING       1.00      0.95      0.98       537

          accuracy                           0.92      2947
         macro avg       0.92      0.92      0.92      2947
      weighted avg       0.92      0.92      0.92      2947
```

13. Summary Questions

Write short answers to the following:

1. Did pruning alone reduce the TensorFlow Lite file size when the pruning wrappers were still attached?
2. Why does `strip_pruning(...)` matter before export?
3. Which model had the smallest file size in this notebook?
4. Did float16 quantization noticeably change the test accuracy?
5. If you were deploying this model on a resource-constrained device, which version would you choose and why?

Summary Answers

1. No, pruning alone did not reduce the TensorFlow Lite file size when the pruning wrappers were still attached.
2. strip_pruning(...) removes the pruning wrappers and masks, leaving only the final pruned weights.
3. The float16 quantized pruned model had the smallest file size.
4. No, float16 quantization did not noticeably change the test accuracy.
5. I would choose the float16 quantized pruned model because it provides the smallest file size while maintaining nearly the same accuracy as the original model.

14. Submission Requirements

Submit the following:

- your completed notebook,
- the generated `.tflite` files,
- output cells or screenshots showing the comparison tables,
- confusion matrices for the baseline model and your final highlighted compressed model,
- and short written observations answering the summary questions.

Make sure your notebook runs from top to bottom without errors using the **`Python (tinyml-arduino)`** kernel.